

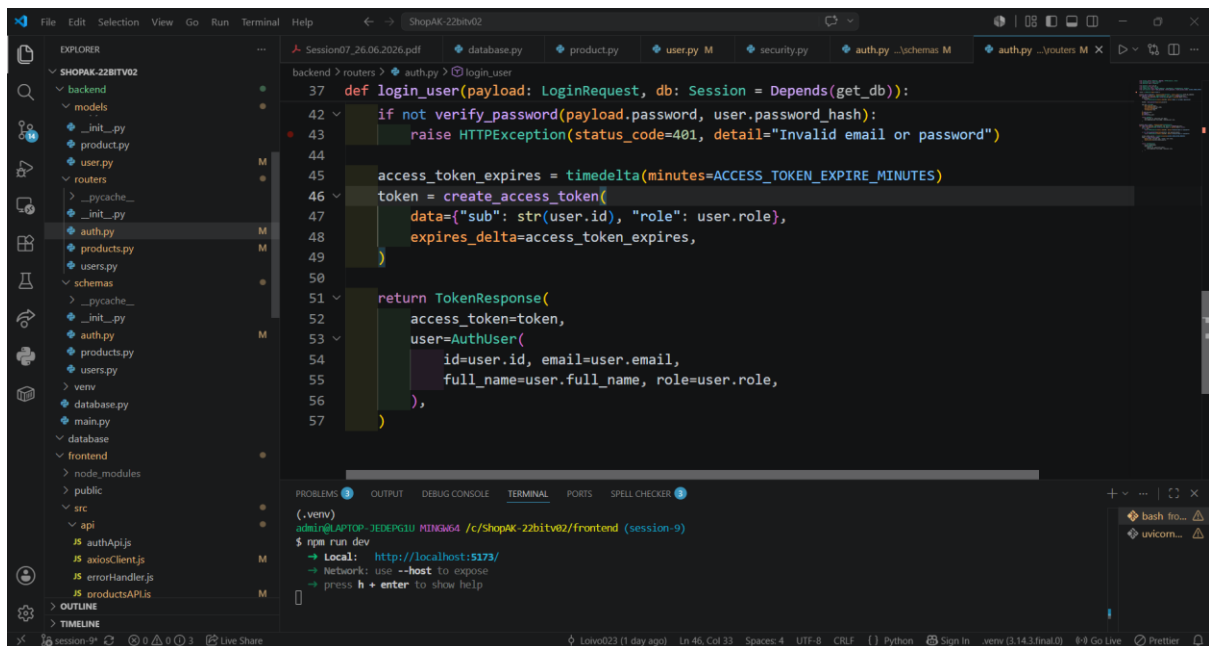
Session10_06.07.2026

Lab CheckList

1. Backend

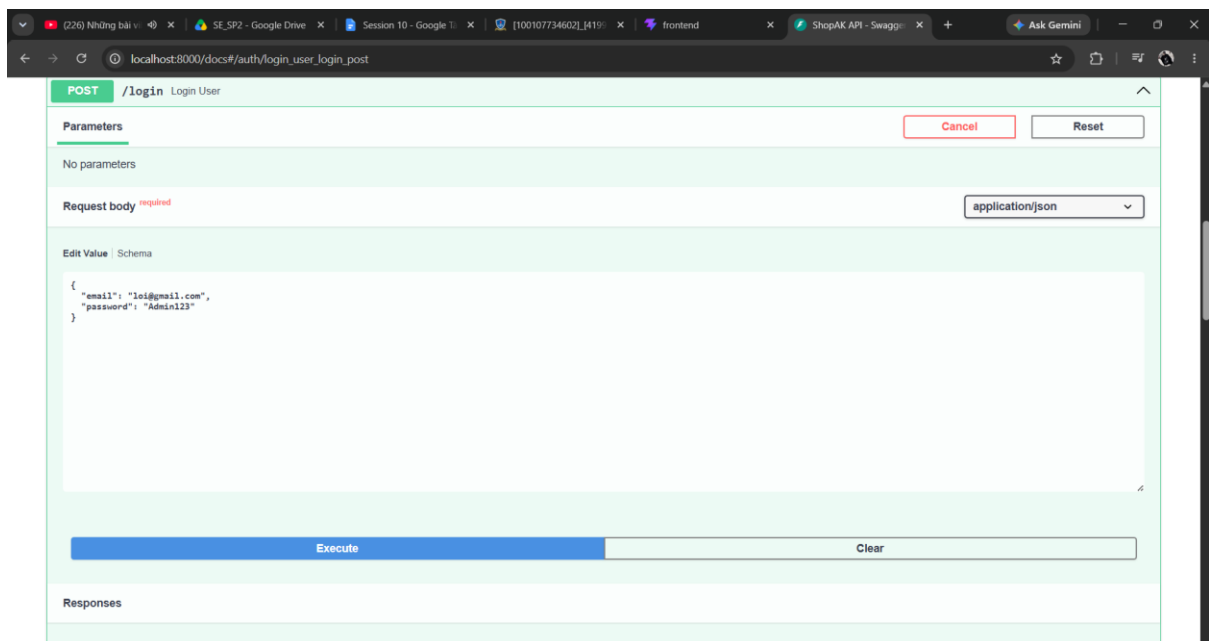
a. JWT chứa trường role

Đây là đoạn code có chứa trường role: “Create_access_token”



```
37 def login_user(payload: LoginRequest, db: Session = Depends(get_db)):
42     if not verify_password(payload.password, user.password_hash):
43         raise HTTPException(status_code=401, detail="Invalid email or password")
44
45     access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
46     token = create_access_token(
47         data={"sub": str(user.id), "role": user.role},
48         expires_delta=access_token_expires,
49     )
50
51     return TokenResponse(
52         access_token=token,
53         user=AuthUser(
54             id=user.id, email=user.email,
55             full_name=user.full_name, role=user.role,
56         ),
57     )
```

Tiếp theo vào swagger POST /login để check response chứa access token với tài khoản admin đã được tạo



The screenshot shows the Swagger UI interface for a REST API. The top navigation bar includes links for "Localhost:8000/docs#/auth/login_user_login_post". The main content area displays details for a specific endpoint:

- Curl**: A cURL command is shown in a dark box:

```
curl -X 'POST' \
'http://localhost:8000/login/' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "email": "loi@gmail.com",
  "password": "Admin123"
}'
```
- Request URL**: `http://localhost:8000/login`
- Server response**: A section indicating the server's response.
- Code**: A table with two columns: "Code" and "Details". It lists a status code of 200.
- Response body**: A JSON object is displayed in a dark box:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IWRkcSIsImVudCI6IjEwMTYlIiwiaWF0IjoiZnV1dGUyOTU0IiwidXNwIjozeWZmZGQkZW50eFQ_djU5AB7deR-91CwoIP2St-ModuxSCR2suquID6stUg",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "email": "loi@gmail.com",
    "full_name": "Admin",
    "role": "ADMIN"
  }
}
```
- Response headers**: A list of headers is shown in a dark box:

```
access-control-allow-credentials: true
content-length: 255
content-type: application/json
date: Sat, 11 Jul 2026 14:34:58 GMT
server: uvicorn
```
- Responses**: A table at the bottom summarizes the responses.

Code	Description	Links
200	Successful Response	No links

Get current user

The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `deps.py` file with the `get_current_user` function. The function is decorated with `@require_admin`. The code logic is as follows:

```
13 def get_current_user(
14     db: Session = Depends(get_db),
15     token: str = Depends(oauth2_scheme),
16 ) -> UserDB:
17     credentials_exception = HTTPException(
18         status_code=status.HTTP_401_UNAUTHORIZED,
19         detail="Could not validate credentials",
20     )
21     try:
22         payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
23         user_id = payload.get("sub")
24         if user_id is None:
25             raise credentials_exception
26     except JWTError:
27         raise credentials_exception
28
29     user = db.query(UserDB).filter(UserDB.id == int(user_id)).first()
30     if user is None:
31         raise credentials_exception
```

The bottom panel shows the terminal with the following output:

```
(.venv)
admin@LAPTOP-JEDEP1U MINGW64 /c/ShopAK-22bitv02 (session-9)
$ uvicorn main:app --reload
INFO: users.email = %(email_1)s
LIMIT %(param_1)s
2026-07-11 21:34:59,357 INFO sqlalchemy.engine.Engine [generated in 0.00662s] ('email_1': 'loi@gmail.com', 'param_1': 1)
INFO: 127.0.0.1:61841 - "POST /login HTTP/1.1" 200 OK
2026-07-11 21:34:59,967 INFO sqlalchemy.engine.Engine ROLLBACK
```

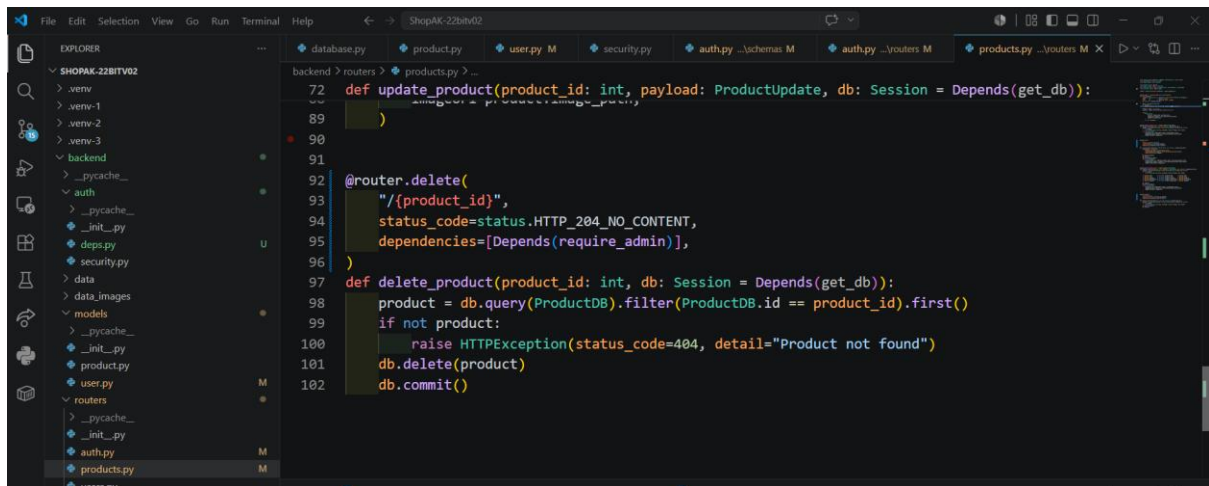
Require_Admin

The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure. The main editor window shows the `deps.py` file with the `require_admin` decorator. The code logic is as follows:

```
13 def get_current_user(
31     raise credentials_exception
32
33     return user
34
35
36 def require_admin(user: UserDB = Depends(get_current_user)) -> UserDB:
37     if user.role != "ADMIN":
38         raise HTTPException(
39             status_code=status.HTTP_403_FORBIDDEN,
40             detail="Admin privileges required",
41         )
42     return user
```

c. Áp dụng vào POST/DELETE products

Bằng cách dùng `require_admin` vào `POST /Products` và `DELETE /Products/{ID}` như trong ảnh

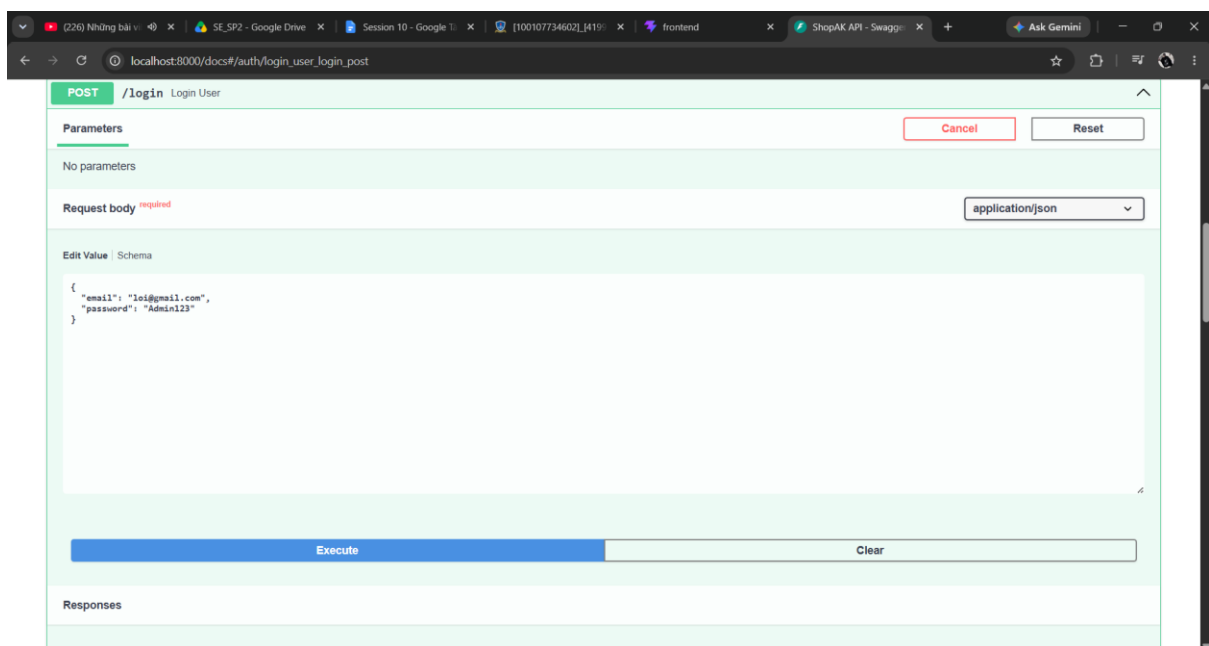


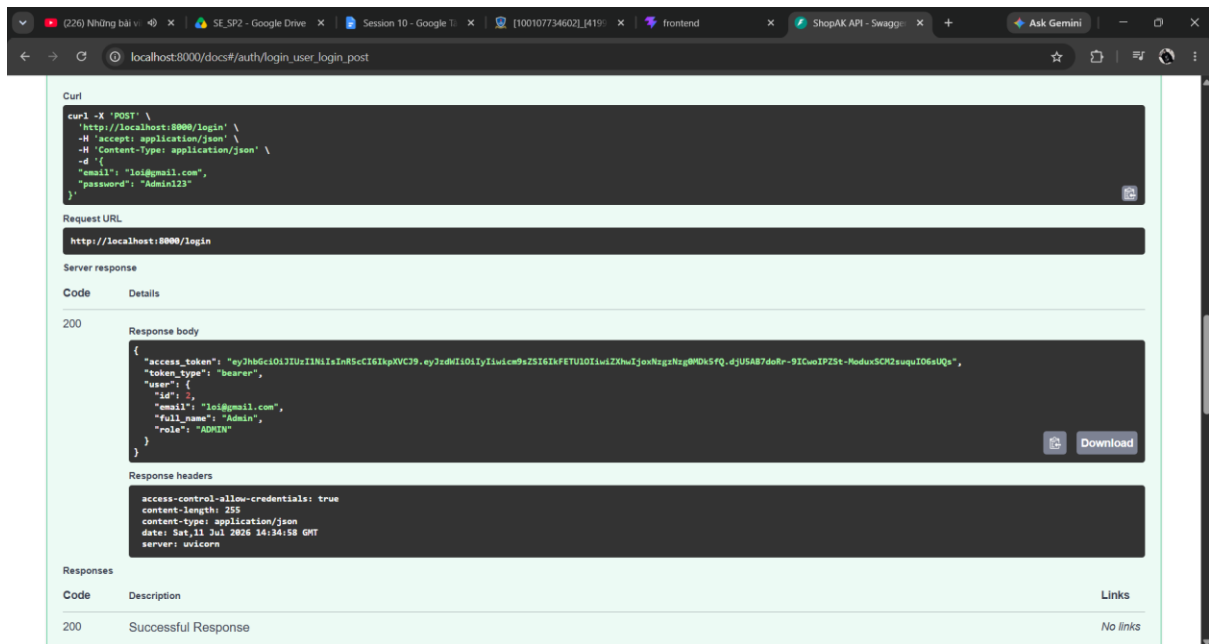
```
72 def update_product(product_id: int, payload: ProductUpdate, db: Session = Depends(get_db)):
89     )
90
91
92 @router.delete(
93     "/{product_id}",
94     status_code=status.HTTP_204_NO_CONTENT,
95     dependencies=[Depends(require_admin)],
96 )
97 def delete_product(product_id: int, db: Session = Depends(get_db)):
98     product = db.query(ProductDB).filter(ProductDB.id == product_id).first()
99     if not product:
100         raise HTTPException(status_code=404, detail="Product not found")
101     db.delete(product)
102     db.commit()
```

d. Test bằng Swagger

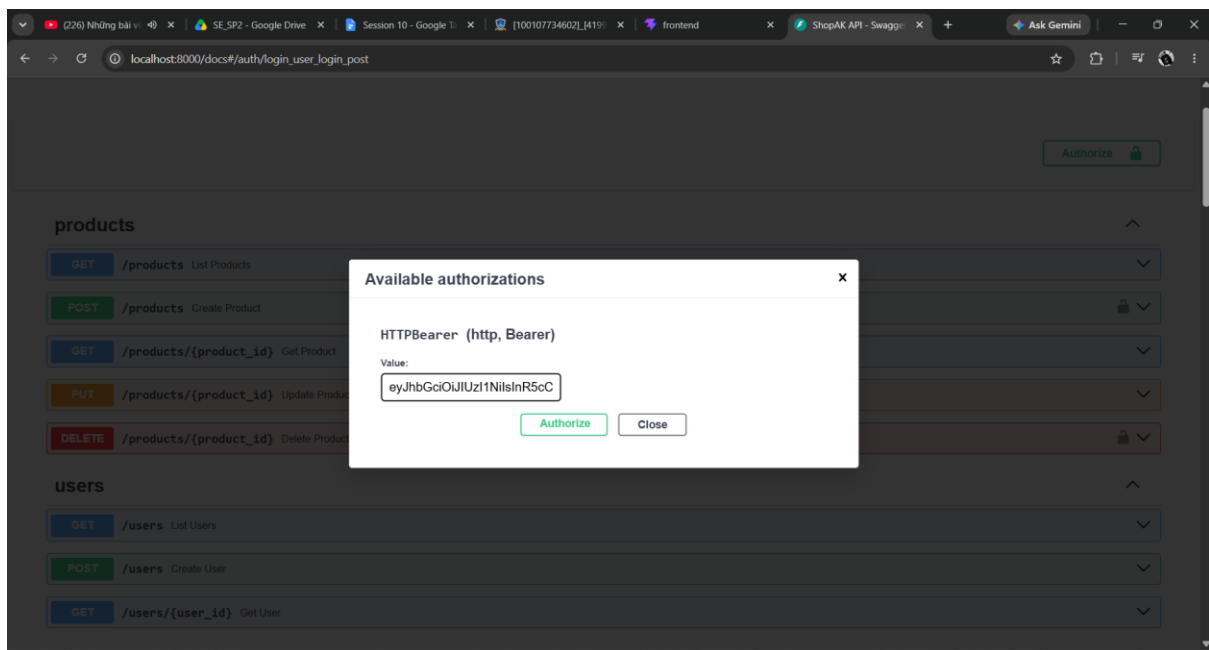
Ở đây chúng ta sẽ test bằng các scenario như sau:

Scenario: Login /Admin với tài khoản của admin thành công sẽ trả về kết quả 200 có access_token

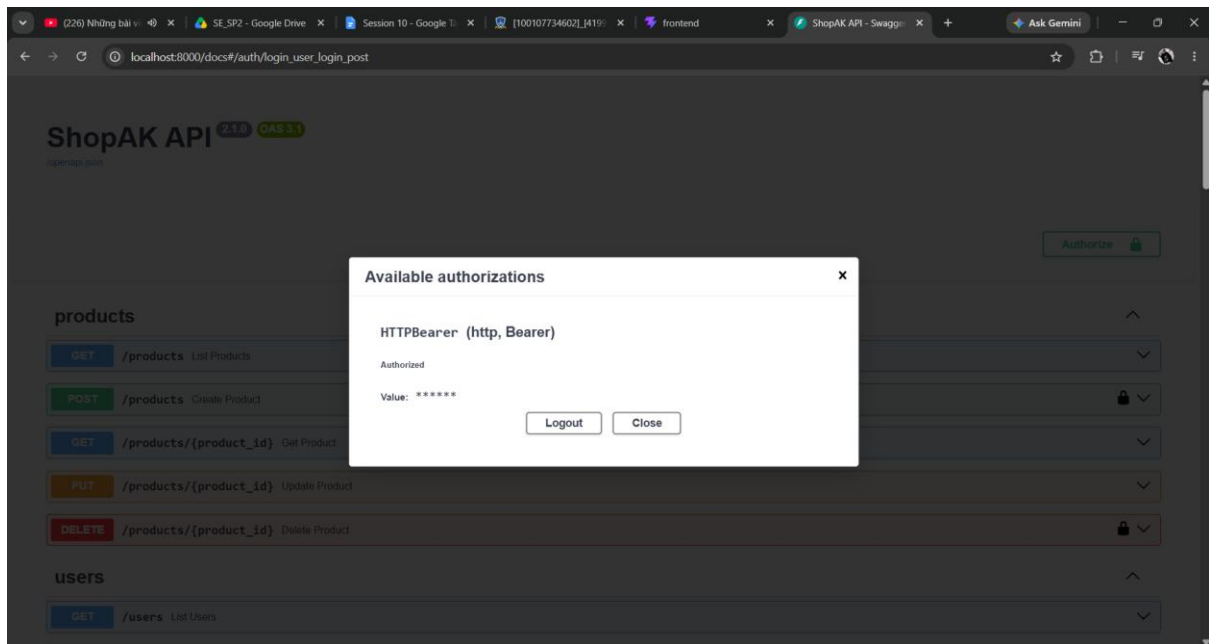




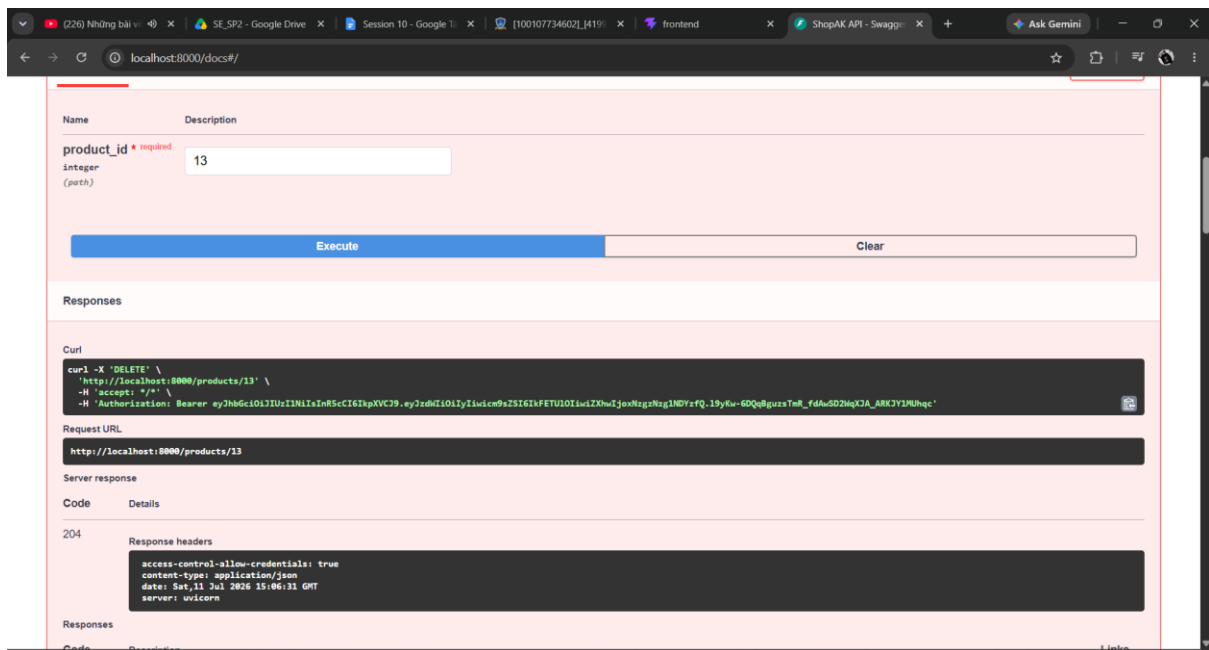
Scenario: ADMIN -> CREATE bằng POST /Products authorize thành công sẽ trả về kết quả 201



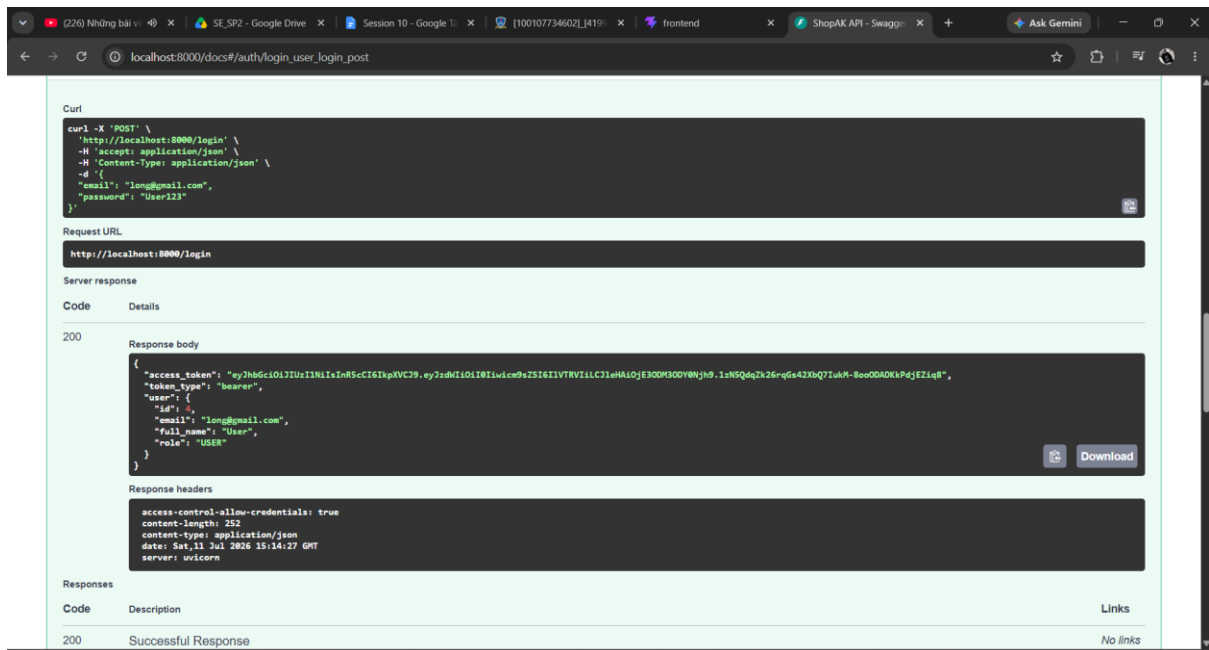
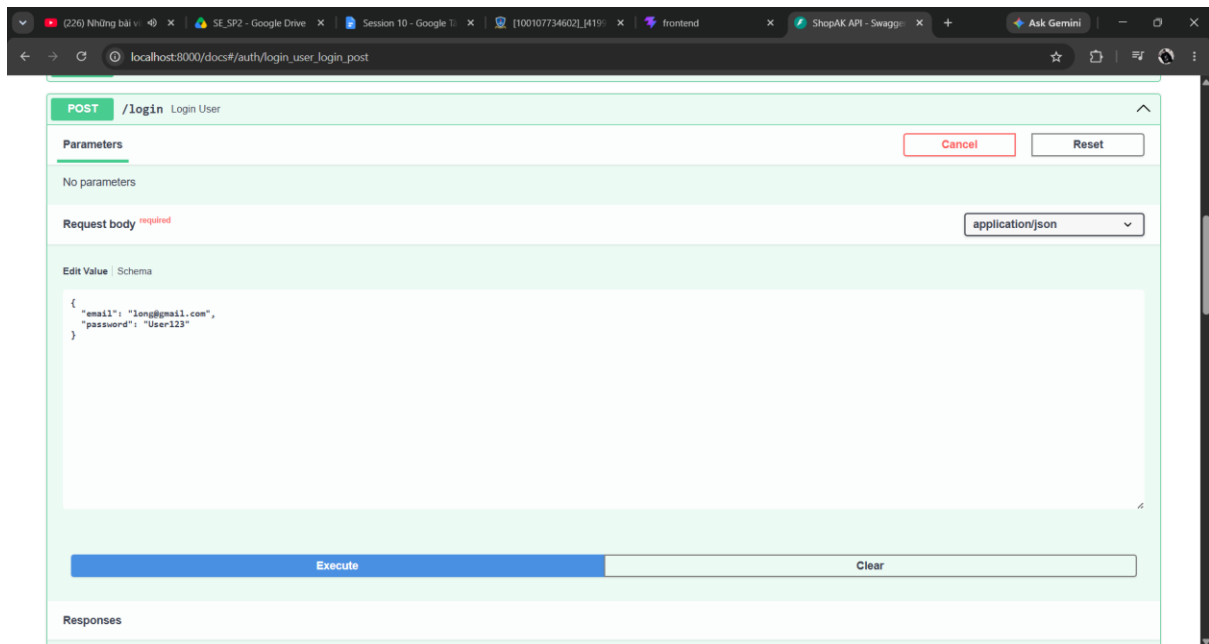
Giờ mọi request (POST /products, DELETE /products/{id}...) sẽ tự động gửi kèm token này



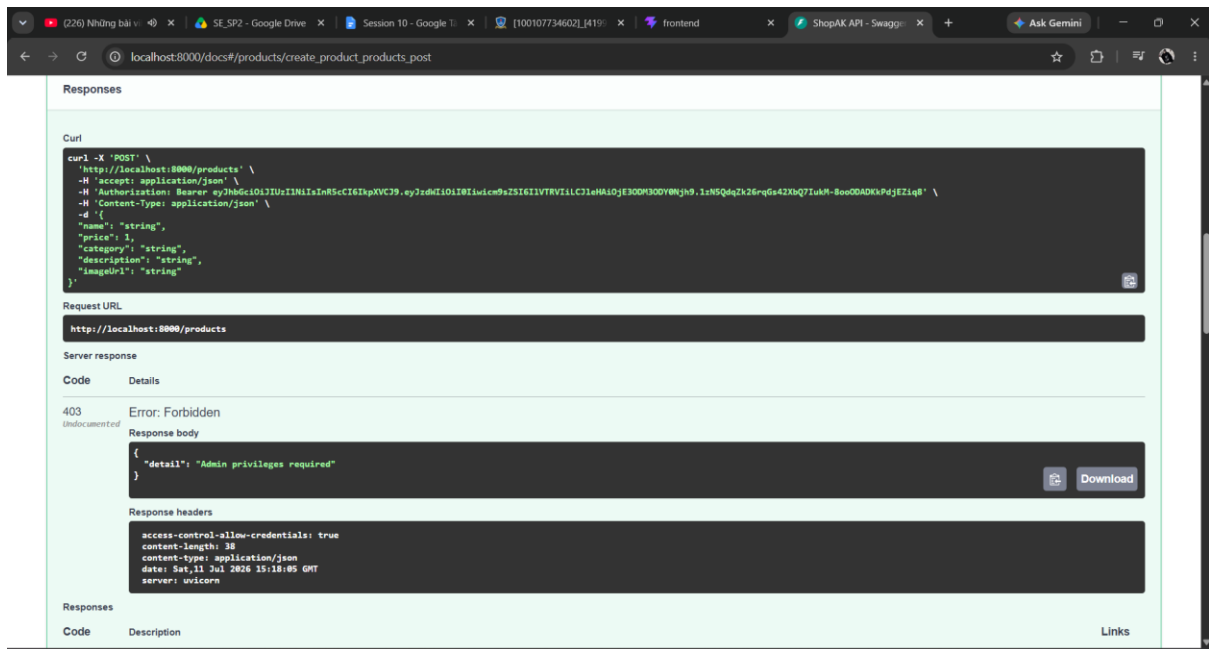
Scenario: Admin -> Delete bằng DELETE /Products/{id} thành công sẽ trả về kết quả response 204



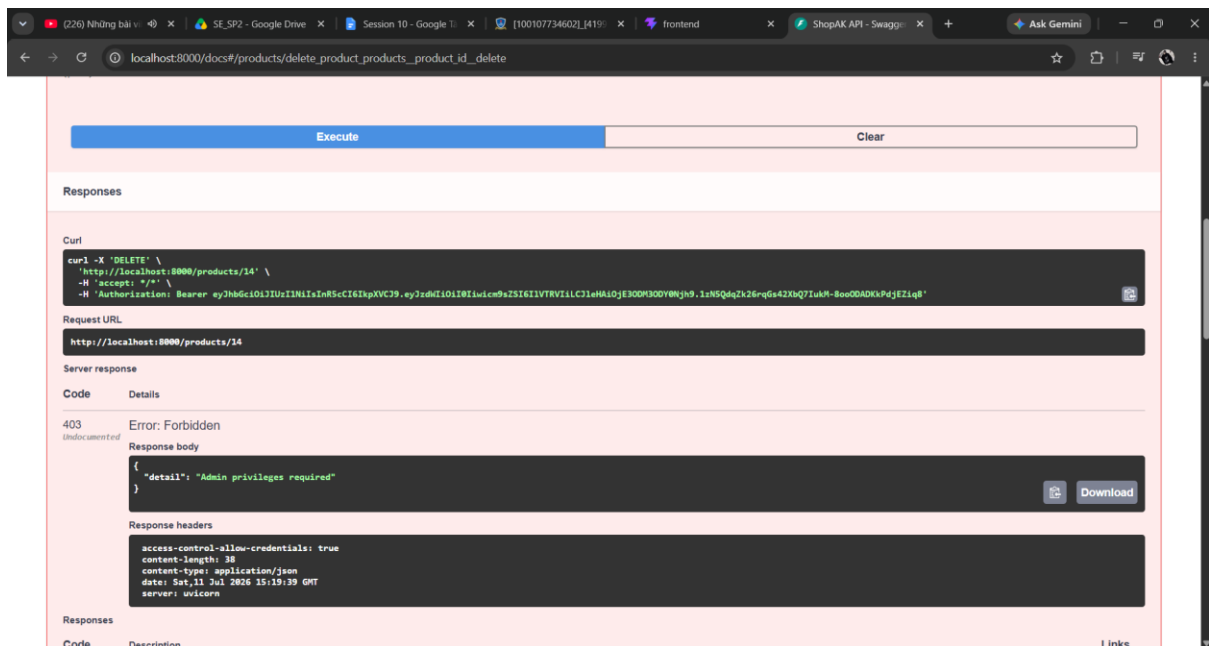
Scenario: Login của Customer với tài khoản của customer đầu tiên lấy token từ POST /Login



Scenario: (CUSTOMER -> Create) sử dụng token user trong Authorize sau đó thử tạo POST/ Products sẽ trả về kết quả 403 không thành công do cần quyền Admin



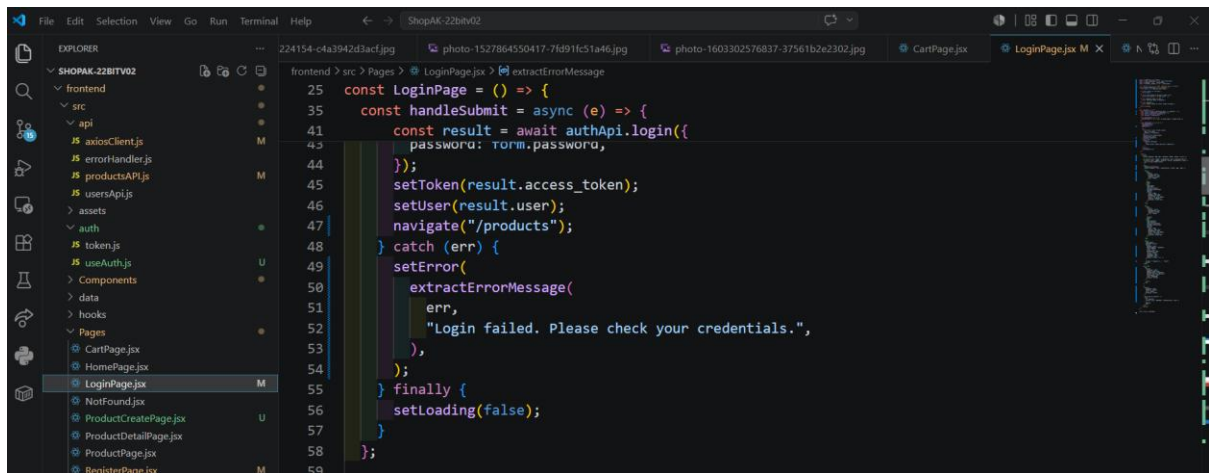
Scenario: (Customer ->Delete) tương tự với delete /products{id} sẽ trả về kết quả là 403 không thành công do cần quyền Admin



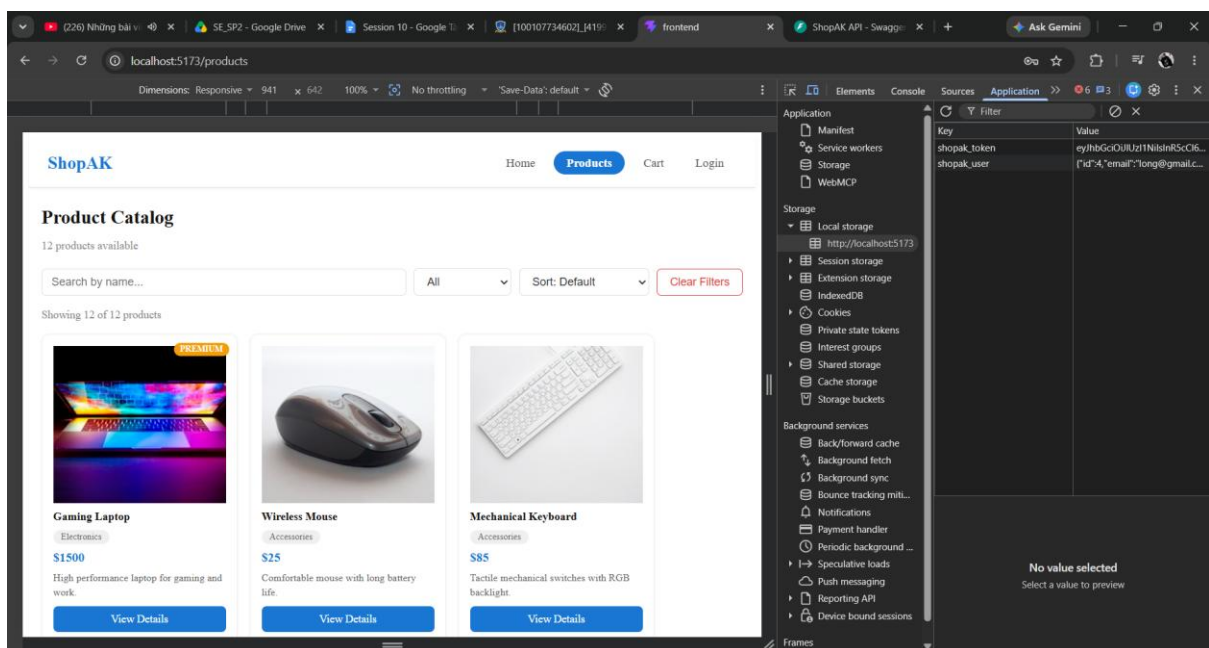
2. Frontend

a. Lưu user vào LocalStorage

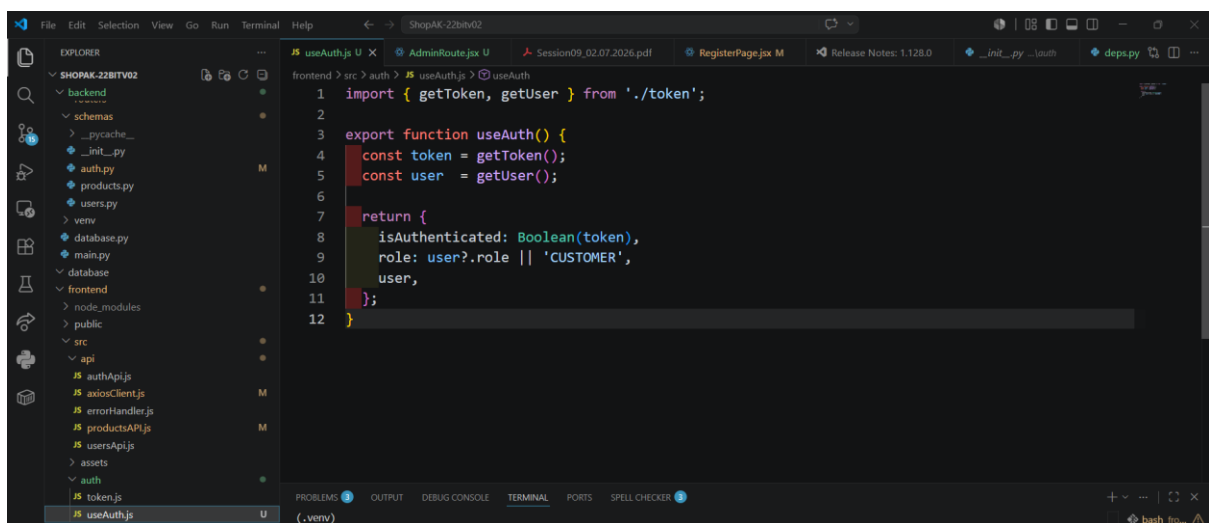
Ta sẽ dùng setToken() và setUser()



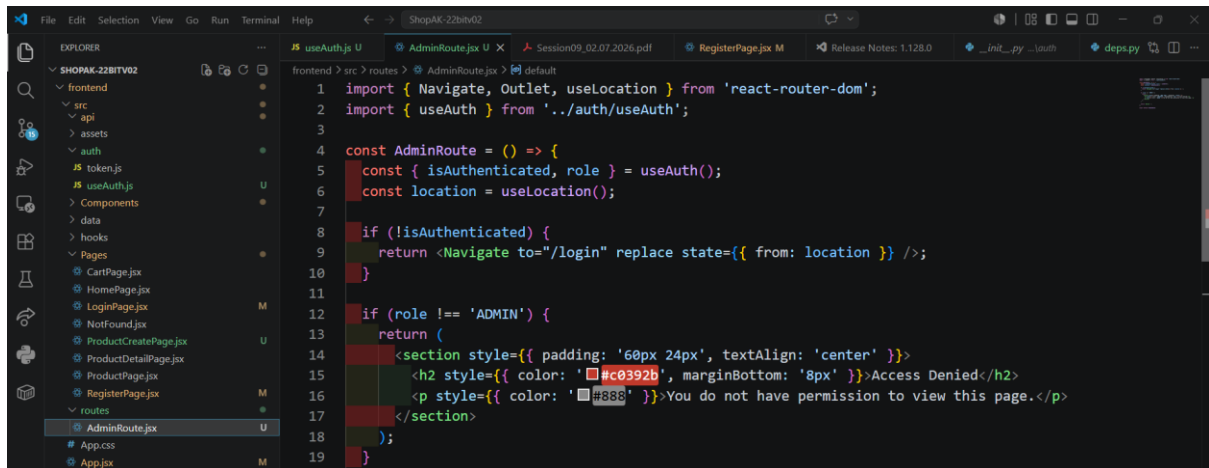
Vào local storage của Application sẽ thấy shopak_token và shopak_user



b. Useauth() hook

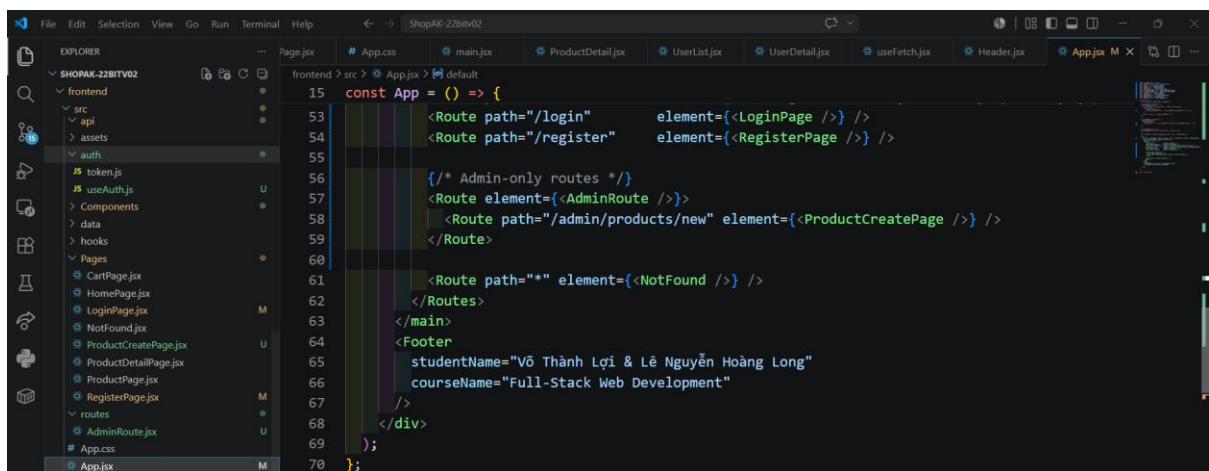


c. AdminRoute.jsx



```
1 import { Navigate, Outlet, useLocation } from 'react-router-dom';
2 import { useAuth } from '../auth/useAuth';
3
4 const AdminRoute = () => {
5   const { isAuthenticated, role } = useAuth();
6   const location = useLocation();
7
8   if (!isAuthenticated) {
9     return <Navigate to="/login" replace state={{ from: location }} />;
10  }
11
12  if (role !== 'ADMIN') {
13    return (
14      <section style={{ padding: '60px 24px', textAlign: 'center' }}>
15        <h2 style={{ color: '#c0392b', marginBottom: '8px' }}>Access Denied</h2>
16        <p style={{ color: '#888' }}>You do not have permission to view this page.</p>
17      </section>
18    );
19  }
```

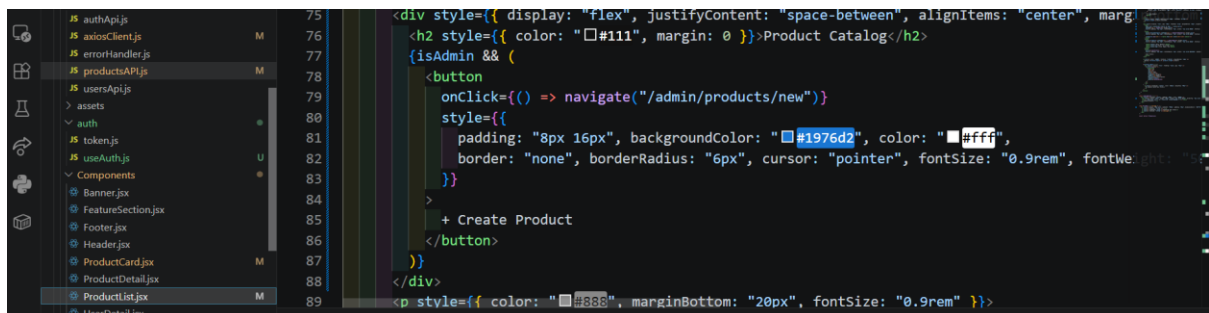
App.jsx bao gồm admin route



```
15 const App = () => {
16   <Routes>
17     <Route path="/login" element={<LoginPage />} />
18     <Route path="/register" element={<RegisterPage />} />
19
20     /* Admin-only routes */
21     <Route element={<AdminRoute />} />
22     <Route path="/admin/products/new" element={<ProductCreatePage />} />
23   </Routes>
24   <Route path="*" element={<NotFound />} />
25 </main>
26 <Footer>
27   studentName="Võ Thành Lợi & Lê Nguyễn Hoàng Long"
28   courseName="Full-Stack Web Development"
29 </Footer>
30 </div>
31 </App>
32 </div>
```

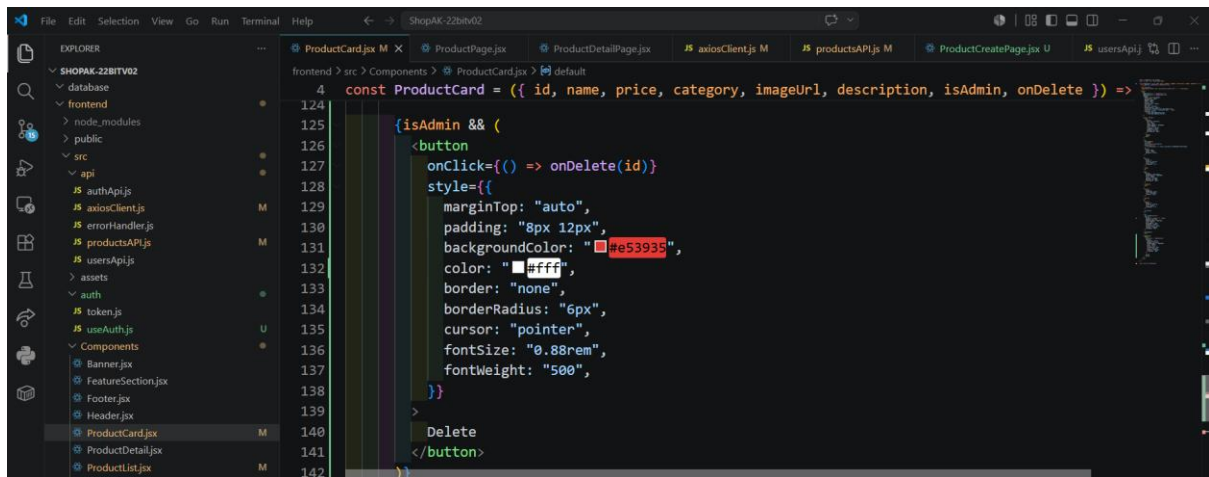
d. Ấn nút theo role

Ấn nút “Create Product” khi role Admin



```
75 <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', margin: '20px 0' }}>
76   <h2 style={{ color: '#111', margin: 0 }}>Product Catalog</h2>
77   {isAdmin && (
78     <button
79       onClick={() => navigate("/admin/products/new")}
80       style={{
81         padding: "8px 16px", backgroundColor: "#1976d2", color: "white",
82         border: "none", borderRadius: "6px", cursor: "pointer", fontSize: "0.9rem", fontWeight: "500"
83       }}
84     >
85       + Create Product
86     </button>
87   )}
88 </div>
89 <p style={{ color: "#888", marginBottom: "20px", fontSize: "0.9rem" }}>
```

Ấn nút “Delete” khi role Admin

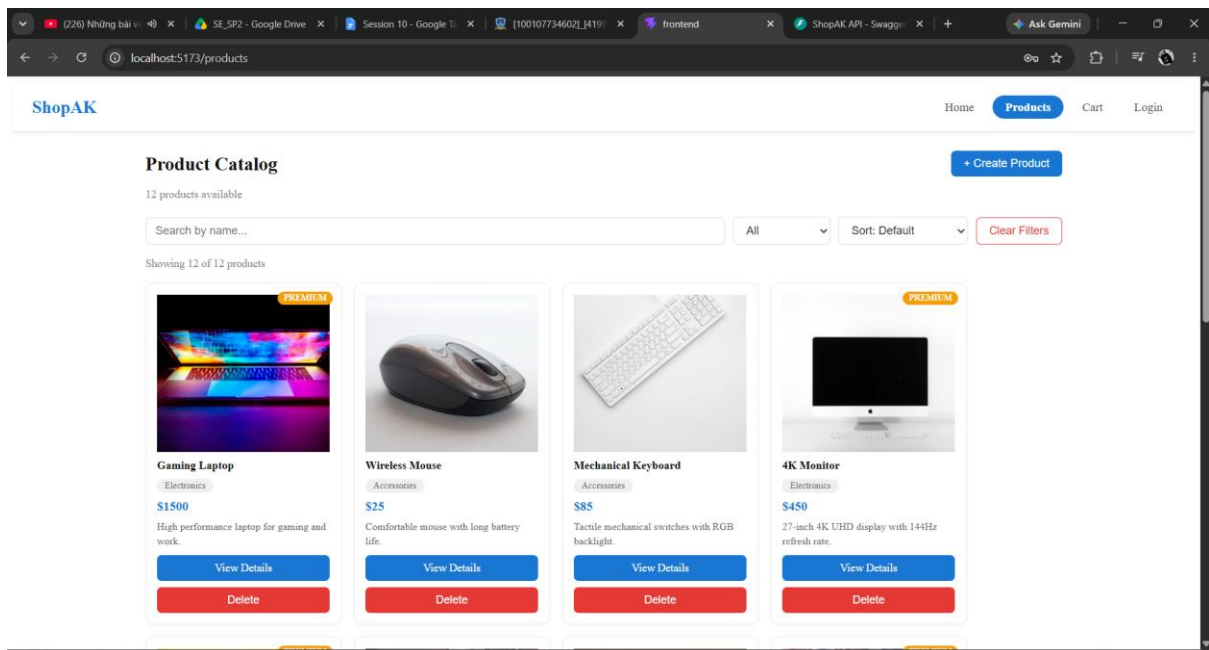


```
4 const ProductCard = ({ id, name, price, category, imageUrl, description, isAdmin, onDelete }) => {
124
125   {isAdmin && (
126     <button
127       onClick={() => onDelete(id)}
128       style={{
129         marginTop: "auto",
130         padding: "8px 12px",
131         backgroundColor: "#e53935",
132         color: "white",
133         border: "none",
134         borderRadius: "6px",
135         cursor: "pointer",
136         fontSize: "0.88rem",
137         fontWeight: "500",
138       }}
139     >
140       Delete
141     </button>
142   )}
```

3. Verification

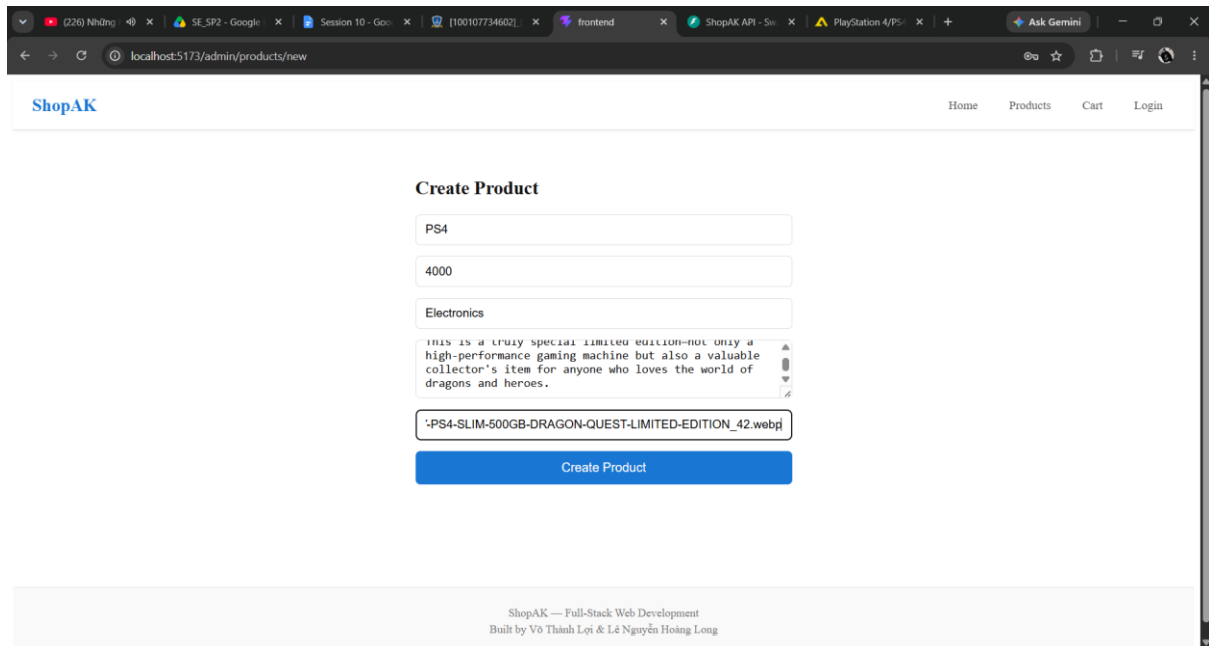
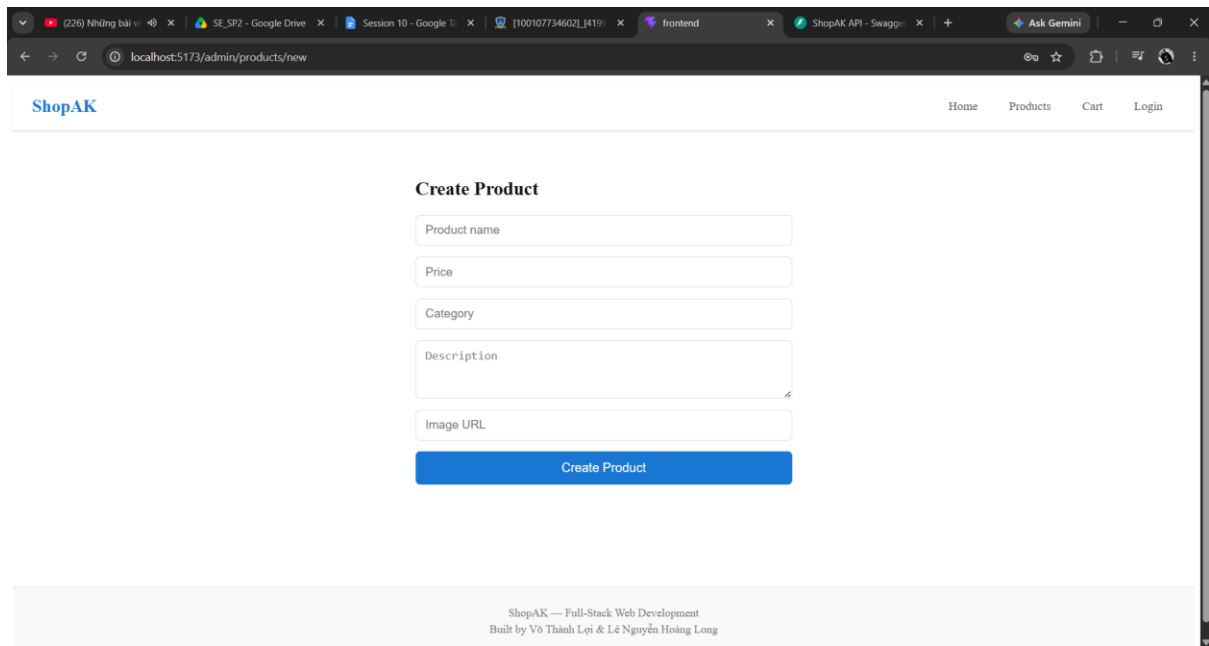
a. Đăng nhập bằng tài khoản ADMIN:

Requirement: Thấy nút “+ Create Product”.

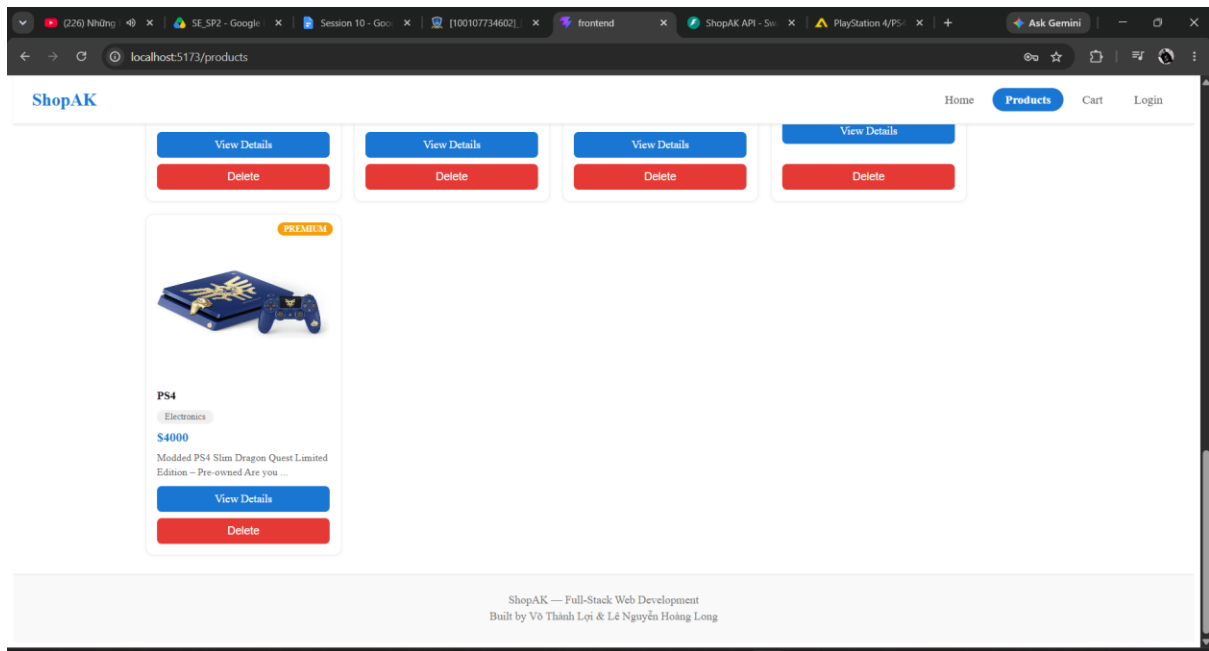


Requirement: Vào /admin/products/new được.

Bây giờ chúng ta sẽ tạo mới sản phẩm

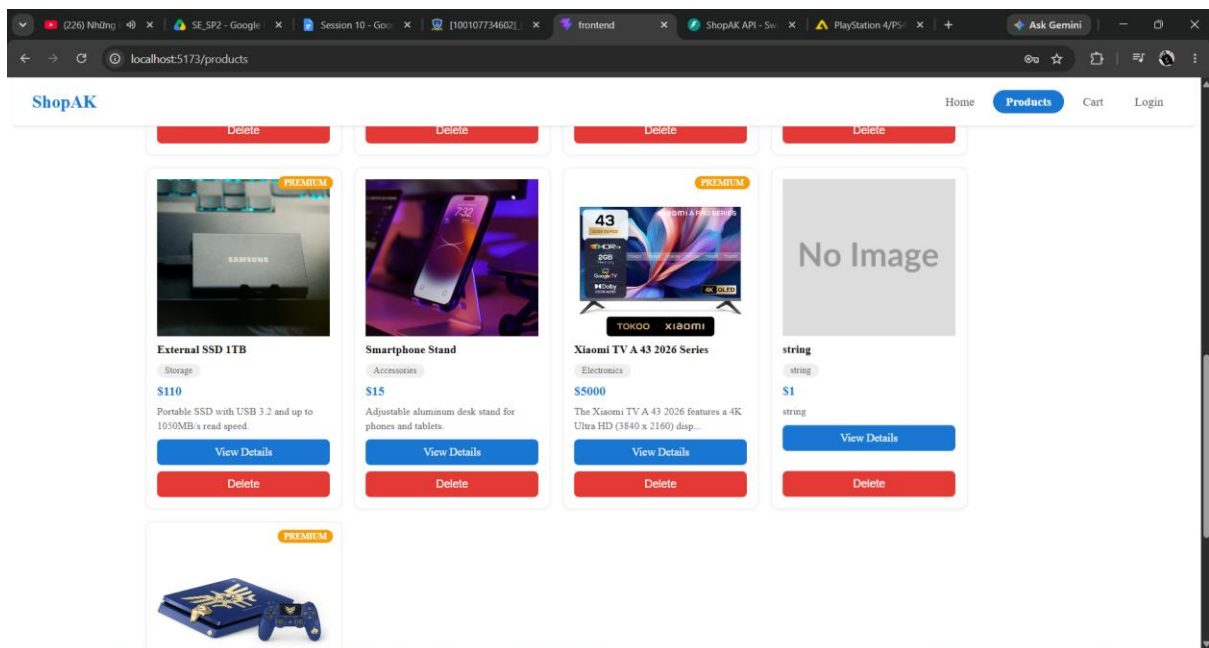


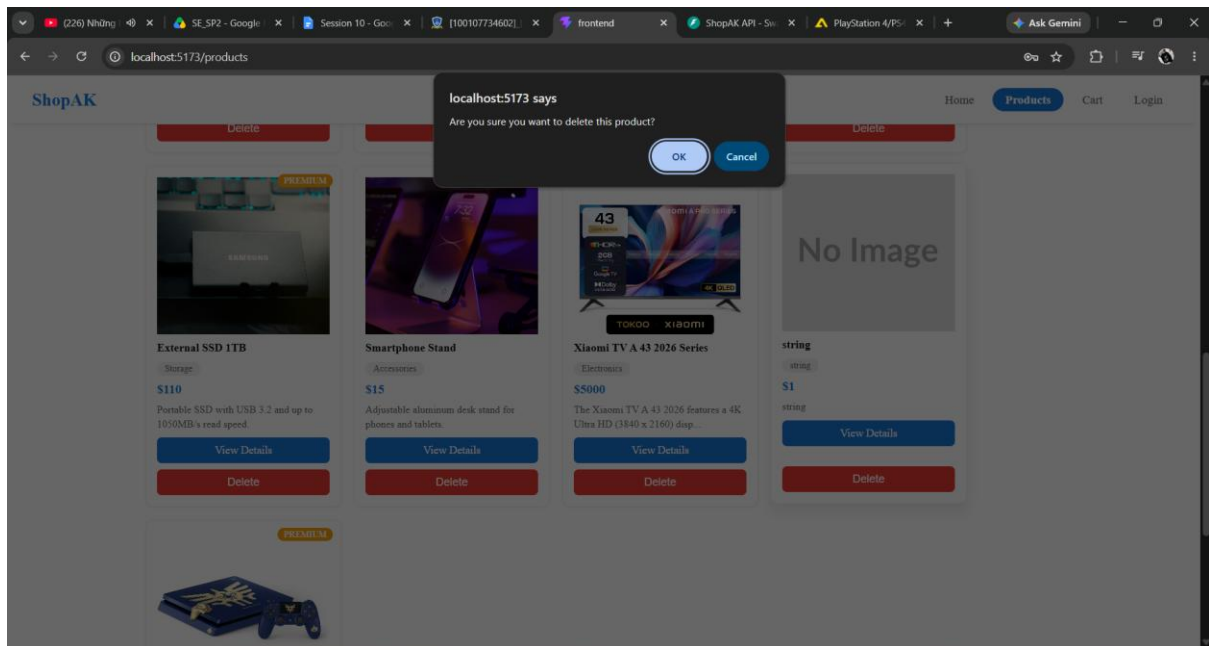
Sau khi nhập thành công sản phẩm trang product sẽ thêm sản phẩm đã tạo:



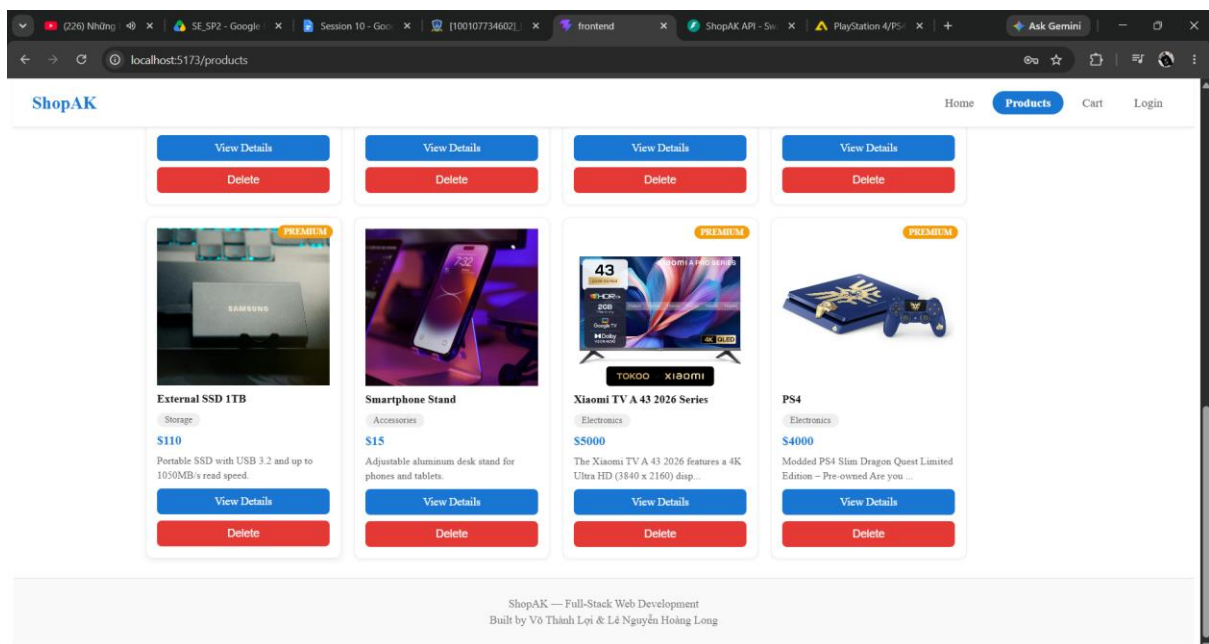
Requirement: Xóa được sản phẩm.

Bây giờ chúng ta sẽ xóa phẩm string





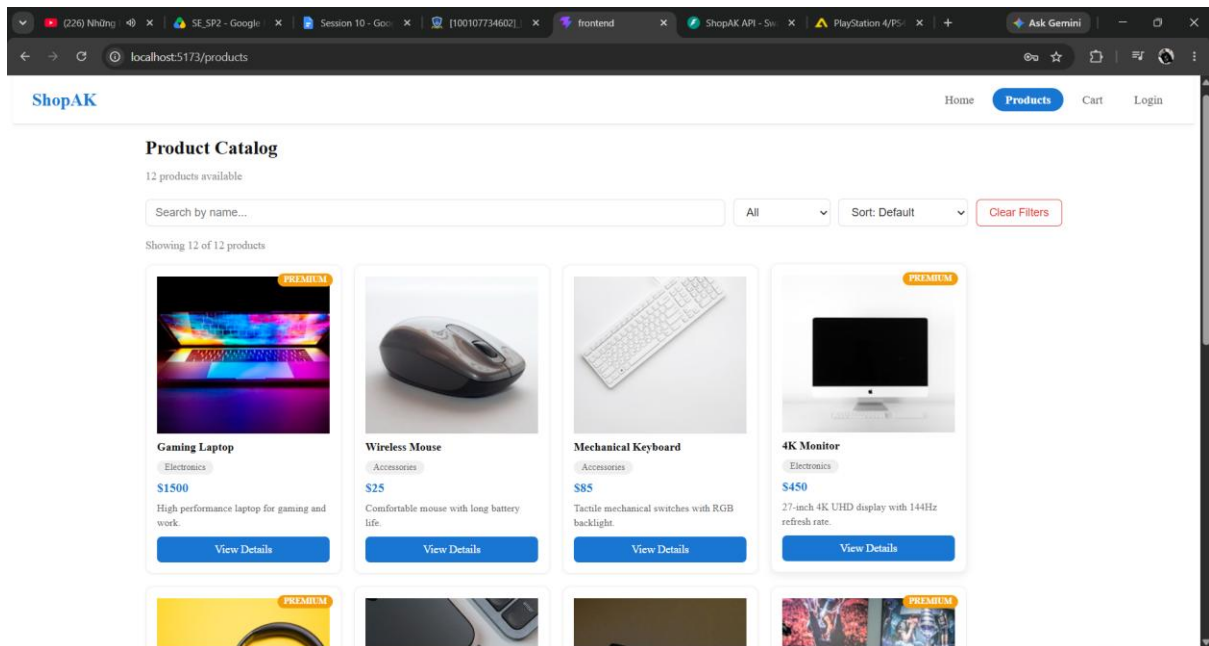
Bây giờ sản phẩm string đã biến mất đưa sản phẩm ps4 lên trên kế bên sản phẩm xiaomi TV



b. Đăng nhập bằng tài khoản CUSTOMER:

Requirement: Không thấy nút “+ Create Product”.

Requirement: Không thấy nút Delete.



Requirement: Truy cập thẳng /admin/products/new báo “Access Denied” hoặc bị redirect về /login.

Khi ta cố truy cập vào /admin/products/new sẽ hiện thông báo bị từ chối:

